DevOps / Cloud Engineering Interview Q&A

Structured technical answers covering AWS, Azure, Jenkins, Terraform, Kubernetes, Docker, and CI/CD practices.

Q1. Some of the applications are in AWS and some are on Azure. How can you achieve communication between these two?

Cross-cloud connectivity options, from most to least common in production setups:

- **Site-to-Site VPN:** Configure an AWS Virtual Private Gateway (VGW) / Transit Gateway and an Azure VPN Gateway, then establish an IPsec tunnel between them. This is the most common low-cost approach for moderate traffic.
- **Dedicated private circuit:** Use AWS Direct Connect paired with Azure ExpressRoute via a shared colocation/partner exchange (e.g., Megaport, Equinix) for low-latency, high-throughput, non-internet-routed traffic.
- **Cloud-agnostic networking overlay:** Tools like Aviatrix, Megaport Cloud Router, or a self-managed WireGuard/IPsec mesh for multi-cloud transit.
- **Public endpoint over HTTPS/API:** If traffic is limited (e.g., REST/gRPC calls), simply expose services behind a load balancer with TLS, authentication (mTLS/OAuth), and IP allow-listing — avoids VPN complexity but is less secure for sensitive internal traffic.
- **DNS & private resolution:** Use Route 53 Resolver / Azure Private DNS forwarding rules so services can resolve each other's private hostnames across the tunnel.
- **Security:** Lock down with Security Groups (AWS) and NSGs (Azure) on both ends, restrict tunnel traffic to specific subnets/ports, and monitor with VPC Flow Logs / NSG Flow Logs.

Q2. My jobs should run on particular nodes. How can you do that in Jenkins?

Jenkins supports node targeting via labels:

- **Label the node:** In *Manage Jenkins → Nodes → (select node) → Configure*, add a label (e.g., *linux-docker*, *high-memory*).
- **Freestyle job:** Check “Restrict where this project can run” and enter the label expression.
- **Declarative pipeline:** Use the *agent* directive with a label.

```
pipeline {
    agent { label 'high-memory' }
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
}
```

- **Scripted pipeline / per-stage targeting:** Use *node('label') { ... }* blocks to pin individual stages to different nodes.
- **Label expressions:** Combine logic, e.g. *linux && docker* or *!windows*, for more granular control.

Q3. What is the default port number of Jenkins? Is it changeable? If yes, how?

- **Default port:** 8080.

- **Changeable:** Yes.

Methods to change it depend on installation type:

- **Linux (Debian/Ubuntu, .deb install):** Edit */etc/default/jenkins* and change *HTTP_PORT=8080* to the desired port, then restart the service.
- **Linux (RedHat/CentOS, .rpm install):** Edit */etc/sysconfig/jenkins*, update *JENKINS_PORT*, then restart.
- **systemd service:** Edit */usr/lib/systemd/system/jenkins.service* (or override file) and update the *--httpPort=* argument, then run *systemctl daemon-reload* and restart Jenkins.
- **WAR file (manual run):** Pass the flag directly: *java -jar jenkins.war --httpPort=9090*.
- **Docker:** Map a different host port at runtime, e.g. *docker run -p 9090:8080 jenkins/jenkins*.
- **Via UI (post-start):** Not directly changeable from the web UI — it must be set at the process/service level before/at startup.

Q4. Can S3 be used to install an application? If yes, how? If no, why?

- **Directly running/installing an application on S3 — No.** S3 is an object storage service, not a compute or execution environment. It cannot execute installers, binaries, or scripts; there is no OS or runtime attached to it.
- **Using S3 as part of an install workflow — Yes.** S3 is commonly used to host and distribute installation artifacts that a compute resource then pulls and executes:
  - Store the installer/package (.zip, .rpm, .deb, JAR, WAR) in an S3 bucket.
  - An EC2 instance's *user-data* script or a configuration tool (Ansible/Chef) runs *aws s3 cp s3://bucket/app.zip .* and then executes the install commands.
  - AWS CodeDeploy natively uses S3 (or GitHub) as the artifact source for deployments to EC2/on-prem instances.
  - Lambda can also pull larger deployment packages/layers from S3 during deployment.

Q5. How can we take backup of particular data in a resource like EC2 at some intervals?

- **AWS Backup:** Create a backup plan with a defined schedule (cron-based), assign EC2 instances/EBS volumes via tags, and set retention/lifecycle rules (e.g., move to cold storage, expire after N days). This is the recommended managed approach.
- **Amazon Data Lifecycle Manager (DLM):** Define a policy directly on EBS volumes/instances to automatically create and prune snapshots at set intervals (e.g., every 12 hours, retain last 7).
- **EventBridge (CloudWatch Events) + Lambda:** Trigger a Lambda function on a cron/rate schedule that calls *create_snapshot* for specific volumes — useful for custom logic (e.g., backing up only a specific mounted directory rather than the whole volume).
- **For file-level/application-level data** (not full volume): Use a cron job inside the instance that tars/zips the target directory and syncs it to S3 (*aws s3 sync*), optionally with versioning and lifecycle policies enabled on the bucket.
- **Cross-region/cross-account resilience:** Enable snapshot copy to another region/account for disaster recovery.

Q6. Write a Terraform script to create an EC2 instance and run a Tomcat server on it, but the script should run Tomcat only ONCE when the EC2 launches.

EC2 *user_data* runs exactly once on first boot by default (cloud-init executes it only during initial instance launch, not on every reboot), which satisfies the “only once” requirement natively — no extra flags needed.

```
terraform {
```

```
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_security_group" "tomcat_sg" {
  name        = "tomcat-sg"
  description = "Allow SSH and Tomcat traffic"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "tomcat_server" {
  ami                    = "ami-0c101f26f147fa7fd"  # Amazon Linux 2 (update per region)
  instance_type          = "t2.micro"
  vpc_security_group_ids = [aws_security_group.tomcat_sg.id]

  # user_data runs ONCE on first boot only (cloud-init default behavior)
  user_data = <<-EOF
              #!/bin/bash
              yum update -y
              yum install -y java-17-amazon-corretto wget

              cd /opt
              wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.89/bin/apache-tomcat-9.0.89.tar.gz
              tar -xzf apache-tomcat-9.0.89.tar.gz
              mv apache-tomcat-9.0.89 tomcat9

              chmod +x /opt/tomcat9/bin/*.sh
              /opt/tomcat9/bin/startup.sh
              EOF

  tags = {
    Name = "tomcat-ec2"
  }
}

output "instance_public_ip" {
  value = aws_instance.tomcat_server.public_ip
}
```

Note: if Tomcat must also auto-start on every subsequent reboot (not just launch), register it as a systemd service inside the same user_data script (separate concern from the “run once” requirement, which user_data already satisfies).

Q7. Explain the Kubernetes deployment strategy used by your project.

Common strategies, with RollingUpdate being the default and most widely used:

- **RollingUpdate (Deployment default):** Pods are replaced incrementally — new ReplicaSet pods are created while old ones are terminated gradually, controlled by *maxSurge* (extra pods allowed above desired count) and *maxUnavailable* (how many can be down during rollout). Ensures zero/near-zero downtime.
- **Recreate:** All old pods are killed before new ones are created. Causes downtime; used when the app cannot run two versions simultaneously (e.g., schema-incompatible releases).
- **Blue-Green:** Two full environments (blue = current, green = new) run in parallel; traffic is switched at the Service/Ingress level once green is validated. Enables instant rollback by switching traffic back.
- **Canary:** A small percentage of traffic is routed to the new version (via multiple Deployments + weighted Service/Ingress, or a service mesh like Istio), gradually increased after validation — reduces blast radius of a bad release.

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

Q8. Code works in one environment but fails in others. Due to what? How do you fix it?

Typical root causes:

- Differing dependency/runtime versions (language, libraries, OS packages) between environments.
- Environment-specific configuration — hardcoded paths, URLs, credentials, or missing environment variables.
- Missing or inconsistent secrets/config files not checked into version control.
- OS/architecture differences (e.g., Linux vs Windows path separators, glibc vs musl, x86 vs ARM).
- Data/state differences — a database schema or seed data present in one environment but not another.
- Network/firewall differences restricting access to dependent services in one environment.

Fixes:

- **Containerize the application** (Docker) so the runtime, dependencies, and OS layer are identical across environments.
- **Infrastructure as Code** (Terraform/CloudFormation) to ensure environment parity at the infra level.
- **12-factor app principles:** externalize config via environment variables, never hardcode environment-specific values.
- **Centralized config/secrets management** (AWS Parameter Store/Secrets Manager, Vault) instead of local files.
- **Consistent CI/CD pipeline:** build the artifact once and promote the same artifact through dev → staging → prod, rather than rebuilding per environment.
- **Automated environment validation** (smoke tests, config-drift detection) before deployment.

Q9. Explain some of the security vulnerabilities you faced in automation.

- **Hardcoded credentials/secrets** in Jenkinsfiles, shell scripts, or Terraform files committed to version control — mitigated by moving to a secrets manager (Vault, AWS Secrets Manager) and Jenkins Credentials Binding.
- **Overly permissive IAM roles** attached to CI/CD agents or EC2 instances (e.g., wildcard *:** actions) — mitigated by enforcing least-privilege, scoped IAM policies per pipeline.
- **Script/Groovy injection in Jenkins pipelines** via unsanitized user-supplied build parameters used directly in *sh* steps — mitigated by input validation and avoiding string interpolation of untrusted input into shell commands.
- **Outdated/vulnerable Jenkins plugins** with known CVEs — mitigated by regular plugin audits and patching cadence.
- **Secrets leaking into build logs** (e.g., printing environment variables for debugging) — mitigated by masking credentials and restricting console log visibility.
- **Vulnerable base Docker images** with outdated OS packages — mitigated by image scanning (Trivy/ECR scanning) integrated into the pipeline as a gating step.
- **Publicly exposed webhook endpoints** without signature verification, enabling spoofed trigger requests — mitigated by validating webhook signatures/tokens.

Q10. Long build times slow down development and deployment. Why does this happen, and how do you address it?

Common causes:

- Monolithic build that compiles/tests the entire codebase even for a small change.
- No dependency or build-layer caching, forcing repeated downloads/recompilation.
- Tests run sequentially instead of in parallel.
- Large, unoptimized Docker images with unnecessary layers being rebuilt.
- Single build agent/node causing queuing instead of distributing load.
- Redundant or unnecessary pipeline steps (e.g., re-running static analysis on unchanged files).

Mitigations:

- Enable build/dependency caching (Maven/Gradle local repo cache, npm/yarn cache, Docker layer caching).
- Parallelize independent stages/tests (e.g., Jenkins *parallel* blocks, splitting test suites across executors).
- Use incremental/modular builds — only rebuild changed modules in a monorepo.
- Adopt multi-stage Docker builds to keep final images lean and reuse cached intermediate layers.
- Scale out with multiple build agents/nodes (or ephemeral cloud agents) to avoid queuing.
- Separate fast unit tests (run on every commit) from slower integration/E2E tests (run less frequently or post-merge).

Q11. Difference between Docker Service, Docker Stack, and Docker Swarm.

Concept	What it is
Docker Swarm	The native clustering/orchestration engine built into Docker. It groups multiple Docker hosts (manager + worker nodes) into a single virtual cluster for scheduling and high availability.
Docker Service	The definition of how a single containerized application should run within a Swarm cluster — image, replica count, ports, update policy, restart policy. Created via *docker service create*.

Docker Stack	A collection of multiple related Services deployed together as one unit, defined in a *docker-compose.yml* file and deployed to Swarm via *docker stack deploy*. It's the Swarm equivalent of *docker-compose up* for multi-service apps.

Hierarchy: Swarm (the cluster) runs Services (individual app definitions), and a Stack groups multiple Services into one deployable, versioned application.

Q12. What is the best way to connect to the portal?

This question is ambiguous without knowing which portal (AWS Console, Azure Portal, Jenkins, an internal app) is meant. General best practices that apply across most enterprise portals:

- **SSO + MFA:** Connect via centralized identity (Azure AD / AWS IAM Identity Center / Okta) with single sign-on and mandatory multi-factor authentication rather than standalone local accounts.
- **Least-privilege, role-based access:** Assign roles/groups scoped to the minimum permissions needed (e.g., IAM roles for AWS Console access, RBAC for Jenkins).
- **Temporary credentials over long-lived keys:** For programmatic access, prefer STS-issued temporary credentials (AssumeRole) over static access keys.
- **Network restriction:** Where the portal is internal (e.g., Jenkins), expose it only over VPN/private endpoint or behind a reverse proxy with TLS, rather than directly on the public internet.
- **Audit logging:** Ensure CloudTrail / Azure Activity Log / Jenkins audit trail is enabled to track who connected and what actions were taken.

Q13. Explain pod affinity and node affinity in Kubernetes.

Node Affinity — controls which *nodes* a pod can be scheduled on, based on node labels:

- *requiredDuringSchedulingIgnoredDuringExecution*: a hard rule — the pod will not be scheduled unless the rule matches.
- *preferredDuringSchedulingIgnoredDuringExecution*: a soft rule — the scheduler tries to honor it but will still schedule the pod elsewhere if needed.

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: disktype
          operator: In
          values: ["ssd"]
```

Pod Affinity / Anti-Affinity — controls placement of a pod *relative to other pods* already running on a node, based on pod labels:

- **Pod affinity:** co-locate pods together on the same node/topology (e.g., place a cache pod on the same node as the app pod that uses it, to reduce latency).
- **Pod anti-affinity:** keep pods apart (e.g., spread replicas of the same Deployment across different nodes/availability zones for high availability).

```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - key: app
          operator: In
          values: ["web"]
```

```
      topologyKey: "kubernetes.io/hostname"
```

Key difference: node affinity matches against node labels; pod affinity/anti-affinity matches against labels of other pods, evaluated within a given *topologyKey* (e.g., hostname, zone).